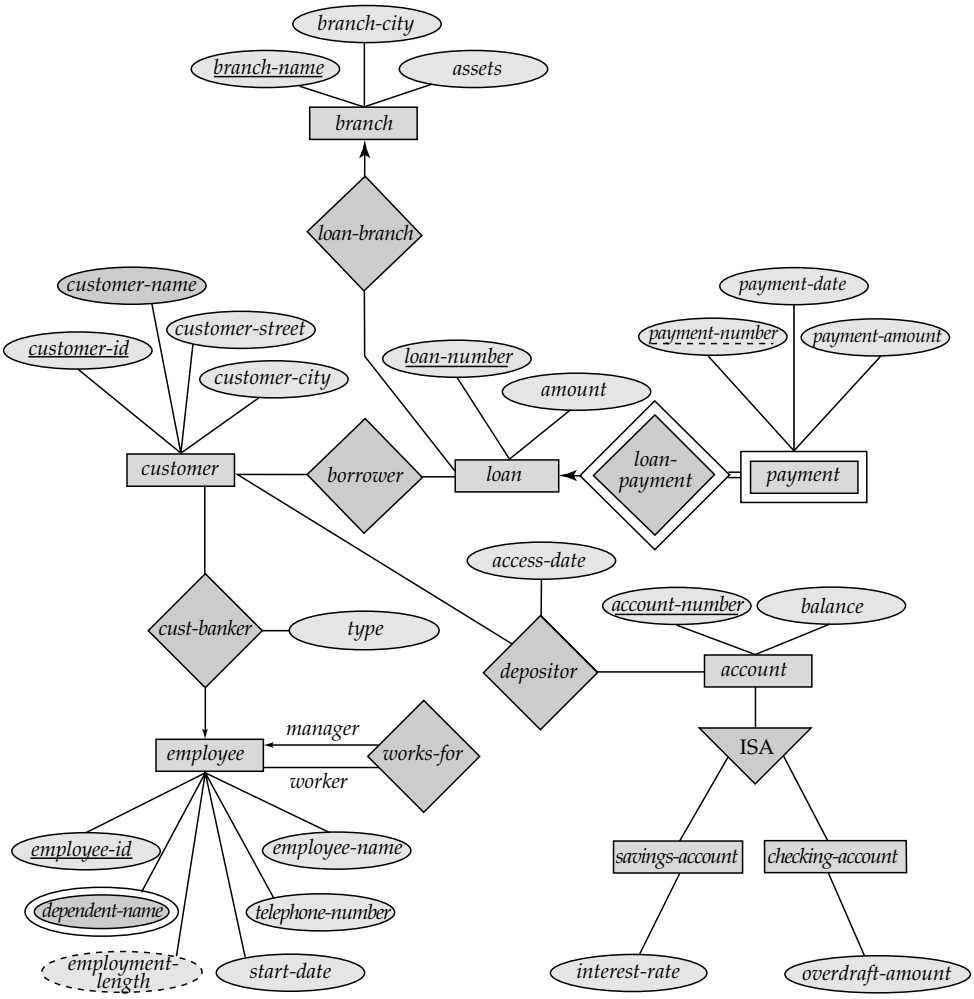


Figure 2.22 E-R diagram for a banking enterprise.

2.9 Reduction of an E-R Schema to Tables

We can represent a database that conforms to an E-R database schema by a collection of tables. For each entity set and for each relationship set in the database, there is a unique table to which we assign the name of the corresponding entity set or relationship set. Each table has multiple columns, each of which has a unique name.

Both the E-R model and the relational-database model are abstract, logical representations of real-world enterprises. Because the two models employ similar design principles, we can convert an E-R design into a relational design. Converting a database representation from an E-R diagram to a table format is the way we arrive at a relational-database design from an E-R diagram. Although important differences

exist between a relation and a table, informally, a relation can be considered to be a table of values.

In this section, we describe how an E-R schema can be represented by tables; and in Chapter 3, we show how to generate a relational-database schema from an E-R schema.

The constraints specified in an E-R diagram, such as primary keys and cardinality constraints, are mapped to constraints on the tables generated from the E-R diagram. We provide more details about this mapping in Chapter 6 after describing how to specify constraints on tables.

2.9.1 Tabular Representation of Strong Entity Sets

Let E be a strong entity set with descriptive attributes $a_1,\ a_2, \ldots, a_n$. We represent this entity by a table called E with n distinct columns, each of which corresponds to one of the attributes of E. Each row in this table corresponds to one entity of the entity set E.

As an illustration, consider the entity set *loan* of the E-R diagram in Figure 2.8. This entity set has two attributes: *loan-number* and *amount*. We represent this entity set by a table called *loan*, with two columns, as in Figure 2.23. The row

(L-17, 1000)

in the *loan* table means that loan number L-17 has a loan amount of $1000. We can add a new entity to the database by inserting a row into a table. We can also delete or modify rows.

Let D_1 denote the set of all loan numbers, and let D_2 denote the set of all balances. Any row of the *loan* table must consist of a 2-tuple $(v_1,\ v_2)$, where v_1 is a loan (that is, v_1 is in set D_1) and v_2 is an amount (that is, v_2 is in set D_2). In general, the *loan* table will contain only a subset of the set of all possible rows. We refer to the set of all possible rows of *loan* as the *Cartesian product* of D_1 and D_2, denoted by

$$D_1 \times D_2$$

In general, if we have a table of n columns, we denote the Cartesian product of $D_1,\ D_2, \cdots, D_n$ by

$$D_1 \times D_2 \times \cdots \times D_{n-1} \times D_n$$

loan-number	*amount*
L-11	900
L-14	1500
L-15	1500
L-16	1300
L-17	1000
L-23	2000
L-93	500

Figure 2.23 The *loan* table.

customer-id	*customer-name*	*customer-street*	*customer-city*
019-28-3746	Smith	North	Rye
182-73-6091	Turner	Putnam	Stamford
192-83-7465	Johnson	Alma	Palo Alto
244-66-8800	Curry	North	Rye
321-12-3123	Jones	Main	Harrison
335-57-7991	Adams	Spring	Pittsfield
336-66-9999	Lindsay	Park	Pittsfield
677-89-9011	Hayes	Main	Harrison
963-96-3963	Williams	Nassau	Princeton

Figure 2.24 The *customer* table.

As another example, consider the entity set *customer* of the E-R diagram in Figure 2.8. This entity set has the attributes *customer-id*, *customer-name*, *customer-street*, and *customer-city*. The table corresponding to *customer* has four columns, as in Figure 2.24.

2.9.2 Tabular Representation of Weak Entity Sets

Let A be a weak entity set with attributes $a_1, a_2, \ldots, a_m$. Let B be the strong entity set on which A depends. Let the primary key of B consist of attributes $b_1, b_2, \ldots, b_n$. We represent the entity set A by a table called A with one column for each attribute of the set:

$$\{a_1, a_2, \ldots, a_m\} \cup \{b_1, b_2, \ldots, b_n\}$$

As an illustration, consider the entity set *payment* in the E-R diagram of Figure 2.16. This entity set has three attributes: *payment-number*, *payment-date*, and *payment-amount*. The primary key of the *loan* entity set, on which *payment* depends, is *loan-number*. Thus, we represent *payment* by a table with four columns labeled *loan-number*, *payment-number*, *payment-date*, and *payment-amount*, as in Figure 2.25.

2.9.3 Tabular Representation of Relationship Sets

Let R be a relationship set, let $a_1, a_2, \ldots, a_m$ be the set of attributes formed by the union of the primary keys of each of the entity sets participating in R, and let the descriptive attributes (if any) of R be $b_1, b_2, \ldots, b_n$. We represent this relationship set by a table called R with one column for each attribute of the set:

$$\{a_1, a_2, \ldots, a_m\} \cup \{b_1, b_2, \ldots, b_n\}$$

As an illustration, consider the relationship set *borrower* in the E-R diagram of Figure 2.8. This relationship set involves the following two entity sets:

- *customer*, with the primary key *customer-id*
- *loan*, with the primary key *loan-number*

loan-number	*payment-number*	*payment-date*	*payment-amount*
L-11	53	7 June 2001	125
L-14	69	28 May 2001	500
L-15	22	23 May 2001	300
L-16	58	18 June 2001	135
L-17	5	10 May 2001	50
L-17	6	7 June 2001	50
L-17	7	17 June 2001	100
L-23	11	17 May 2001	75
L-93	103	3 June 2001	900
L-93	104	13 June 2001	200

Figure 2.25 The *payment* table.

Since the relationship set has no attributes, the *borrower* table has two columns, labeled *customer-id* and *loan-number*, as shown in Figure 2.26.

2.9.3.1 Redundancy of Tables

A relationship set linking a weak entity set to the corresponding strong entity set is treated specially. As we noted in Section 2.6, these relationships are many-to-one and have no descriptive attributes. Furthermore, the primary key of a weak entity set includes the primary key of the strong entity set. In the E-R diagram of Figure 2.16, the weak entity set *payment* is dependent on the strong entity set *loan* via the relationship set *loan-payment*. The primary key of *payment* is {*loan-number*, *payment-number*}, and the primary key of *loan* is {*loan-number*}. Since *loan-payment* has no descriptive attributes, the *loan-payment* table would have two columns, *loan-number* and *payment-number*. The table for the entity set *payment* has four columns, *loan-number*, *payment-number*, *payment-date*, and *payment-amount*. Every (*loan-number*, *payment-number*) combination in *loan-payment* would also be present in the *payment* table, and vice versa. Thus, the *loan-payment* table is redundant. In general, the table for the relationship set

customer-id	*loan-number*
019-28-3746	L-11
019-28-3746	L-23
244-66-8800	L-93
321-12-3123	L-17
335-57-7991	L-16
555-55-5555	L-14
677-89-9011	L-15
963-96-3963	L-17

Figure 2.26 The *borrower* table.

linking a weak entity set to its corresponding strong entity set is redundant and does not need to be present in a tabular representation of an E-R diagram.

2.9.3.2 Combination of Tables

Consider a many-to-one relationship set AB from entity set A to entity set B. Using our table-construction scheme outlined previously, we get three tables: A, B, and AB. Suppose further that the participation of A in the relationship is total; that is, every entity a in the entity set A must participate in the relationship AB. Then we can combine the tables A and AB to form a single table consisting of the union of columns of both tables.

As an illustration, consider the E-R diagram of Figure 2.27. The double line in the E-R diagram indicates that the participation of *account* in the *account-branch* is total. Hence, an account cannot exist without being associated with a particular branch. Further, the relationship set *account-branch* is many to one from *account* to *branch*. Therefore, we can combine the table for *account-branch* with the table for *account* and require only the following two tables:

- *account*, with attributes *account-number*, *balance*, and *branch-name*
- *branch*, with attributes *branch-name*, *branch-city*, and *assets*

2.9.4 Composite Attributes

We handle composite attributes by creating a separate attribute for each of the component attributes; we do not create a separate column for the composite attribute itself. Suppose *address* is a composite attribute of entity set *customer*, and the components of *address* are *street* and *city*. The table generated from *customer* would then contain columns *address-street* and *address-city*; there is no separate column for *address*.

2.9.5 Multivalued Attributes

We have seen that attributes in an E-R diagram generally map directly into columns for the appropriate tables. Multivalued attributes, however, are an exception; new tables are created for these attributes.

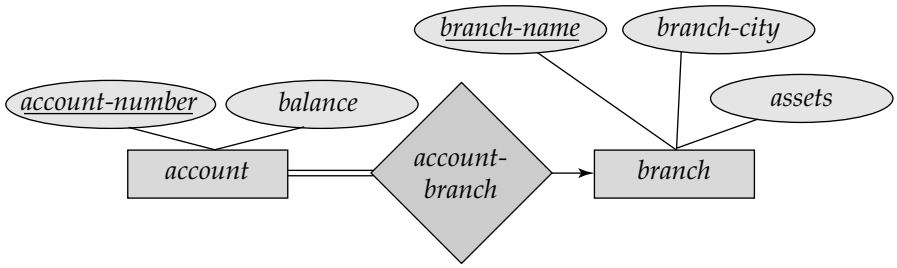


Figure 2.27 E-R diagram.

For a multivalued attribute M, we create a table T with a column C that corresponds to M and columns corresponding to the primary key of the entity set or relationship set of which M is an attribute. As an illustration, consider the E-R diagram in Figure 2.22. The diagram includes the multivalued attribute *dependent-name*. For this multivalued attribute, we create a table *dependent-name*, with columns *dname*, referring to the *dependent-name* attribute of *employee*, and *employee-id*, representing the primary key of the entity set *employee*. Each dependent of an employee is represented as a unique row in the table.

2.9.6 Tabular Representation of Generalization

There are two different methods for transforming to a tabular form an E-R diagram that includes generalization. Although we refer to the generalization in Figure 2.17 in this discussion, we simplify it by including only the first tier of lower-level entity sets—that is, *savings-account* and *checking-account*.

1. Create a table for the higher-level entity set. For each lower-level entity set, create a table that includes a column for each of the attributes of that entity set plus a column for each attribute of the primary key of the higher-level entity set. Thus, for the E-R diagram of Figure 2.17, we have three tables:
   - *account*, with attributes *account-number* and *balance*
   - *savings-account*, with attributes *account-number* and *interest-rate*
   - *checking-account*, with attributes *account-number* and *overdraft-amount*
2. An alternative representation is possible, if the generalization is disjoint and complete—that is, if no entity is a member of two lower-level entity sets directly below a higher-level entity set, and if every entity in the higher level entity set is also a member of one of the lower-level entity sets. Here, do not create a table for the higher-level entity set. Instead, for each lower-level entity set, create a table that includes a column for each of the attributes of that entity set plus a column for *each* attribute of the higher-level entity set. Then, for the E-R diagram of Figure 2.17, we have two tables.
   - *savings-account*, with attributes *account-number*, *balance*, and *interest-rate*
   - *checking-account*, with attributes *account-number*, *balance*, and *overdraft-amount*

   The *savings-account* and *checking-account* relations corresponding to these tables both have *account-number* as the primary key.

If the second method were used for an overlapping generalization, some values such as *balance* would be stored twice unnecessarily. Similarly, if the generalization were not complete—that is, if some accounts were neither savings nor checking accounts—then such accounts could not be represented with the second method.

2.9.7 Tabular Representation of Aggregation

Transforming an E-R diagram containing aggregation to a tabular form is straightforward. Consider the diagram of Figure 2.19. The table for the relationship set

manages between the aggregation of *works-on* and the entity set *manager* includes a column for each attribute in the primary keys of the entity set *manager* and the relationship set *works-on*. It would also include a column for any descriptive attributes, if they exist, of the relationship set *manages*. We then transform the relationship sets and entity sets within the aggregated entity.

2.10 The Unified Modeling Language UML**

Entity-relationship diagrams help model the data representation component of a software system. Data representation, however, forms only one part of an overall system design. Other components include models of user interactions with the system, specification of functional modules of the system and their interaction, etc. The **Unified Modeling Language** (UML), is a proposed standard for creating specifications of various components of a software system. Some of the parts of UML are:

- **Class diagram**. A class diagram is similar to an E-R diagram. Later in this section we illustrate a few features of class diagrams and how they relate to E-R diagrams.
- **Use case diagram**. Use case diagrams show the interaction between users and the system, in particular the steps of tasks that users perform (such as withdrawing money or registering for a course).
- **Activity diagram**. Activity diagrams depict the flow of tasks between various components of a system.
- **Implementation diagram**. Implementation diagrams show the system components and their interconnections, both at the software component level and the hardware component level.

We do not attempt to provide detailed coverage of the different parts of UML here. See the bibliographic notes for references on UML. Instead we illustrate some features of UML through examples.

Figure 2.28 shows several E-R diagram constructs and their equivalent UML class diagram constructs. We describe these constructs below. UML shows entity sets as boxes and, unlike E-R, shows attributes within the box rather than as separate ellipses. UML actually models objects, whereas E-R models entities. Objects are like entities, and have attributes, but additionally provide a set of functions (called methods) that can be invoked to compute values on the basis of attributes of the objects, or to update the object itself. Class diagrams can depict methods in addition to attributes. We cover objects in Chapter 8.

We represent binary relationship sets in UML by just drawing a line connecting the entity sets. We write the relationship set name adjacent to the line. We may also specify the role played by an entity set in a relationship set by writing the role name on the line, adjacent to the entity set. Alternatively, we may write the relationship set name in a box, along with attributes of the relationship set, and connect the box by a dotted line to the line depicting the relationship set. This box can then be treated as

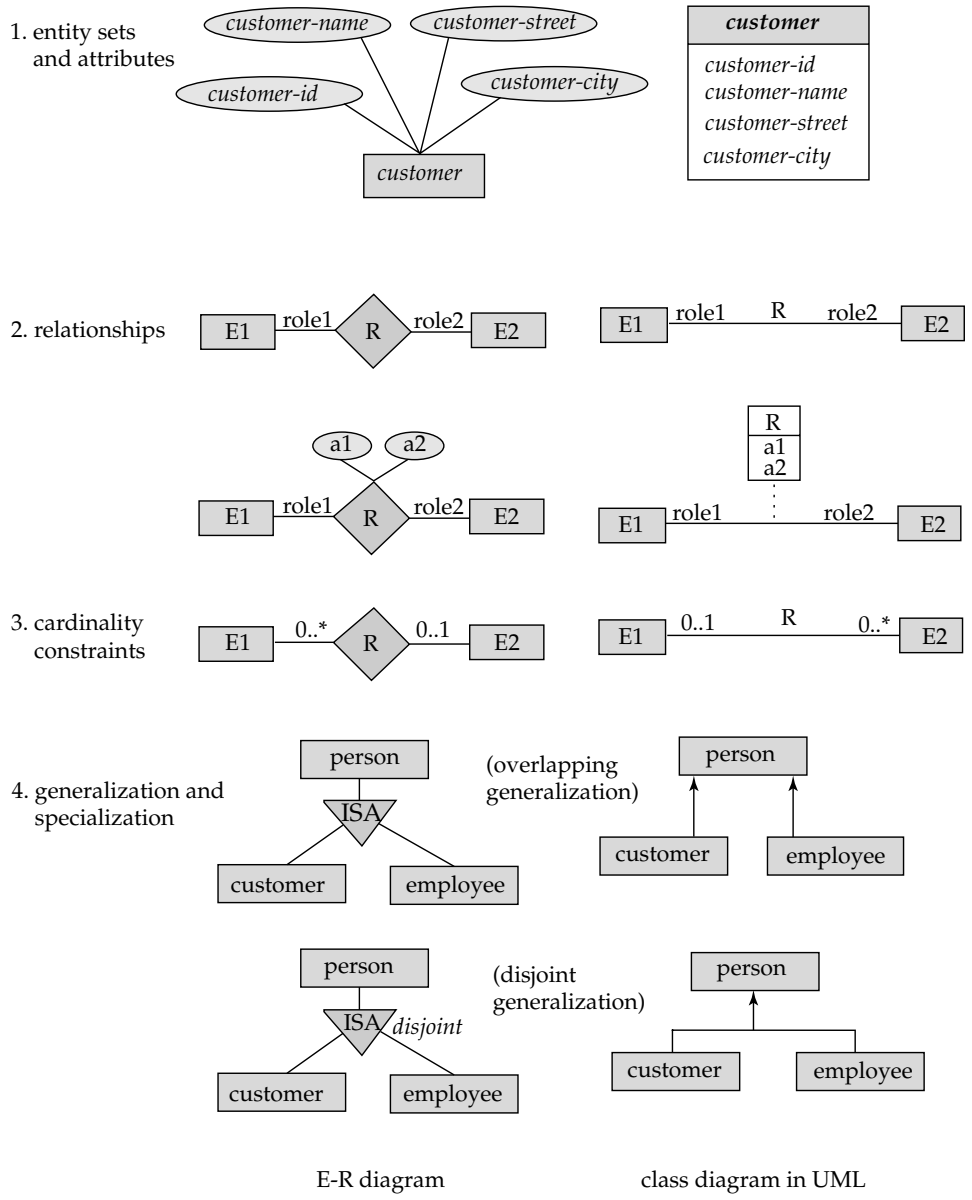


Figure 2.28 Symbols used in the UML class diagram notation.

an entity set, in the same way as an aggregation in E-R diagrams and can participate in relationships with other entity sets.

Nonbinary relationships cannot be directly represented in UML—they have to be converted to binary relationships by the technique we have seen earlier in Section 2.4.3.